Data.Char: https://hackage.haskell.org/package/base/docs/Data-Char.html
Data.String: https://hackage.haskell.org/package/base/docs/Data-String.html

```
-- Data.Char
data Char

toUpper :: Char -> Char
toLower :: Char -> Char
toTitle :: Char -> Char
digitToInt :: Char -> Int
intToDigit :: Int -> Char
ord :: Char -> Int
chr :: Int -> Char
showLitChar :: Char -> ShowS
lexLitChar :: ReadS String
readLitChar :: ReadS Char

isControl :: Char -> Bool
isSpace :: Char -> Bool
isLower :: Char -> Bool
isLowerCase :: Char -> Bool
isUpper :: Char -> Bool
isUpperCase :: Char -> Bool
isAlpha :: Char -> Bool
isAlphaNum :: Char -> Bool
isPrint :: Char -> Bool
isDigit :: Char -> Bool
isOctDigit :: Char -> Bool
isHexDigit :: Char -> Bool
isLetter :: Char -> Bool
isMark :: Char -> Bool
isNumber :: Char -> Bool
isPunctuation :: Char -> Bool
isSymbol :: Char -> Bool
isSeparator :: Char -> Bool
isAscii :: Char -> Bool
isLatin1 :: Char -> Bool
isAsciiUpper :: Char -> Bool
isAsciiLower :: Char -> Bool


data GeneralCategory
  = UppercaseLetter
  | LowercaseLetter
  | TitlecaseLetter
  | ModifierLetter
  | OtherLetter
  | NonSpacingMark
  | SpacingCombiningMark
  | EnclosingMark
  | DecimalNumber
```

```
  | LetterNumber
  | OtherNumber
  | ConnectorPunctuation
  | DashPunctuation
  | OpenPunctuation
  | ClosePunctuation
  | InitialQuote
  | FinalQuote
  | OtherPunctuation
  | MathSymbol
generalCategory :: Char -> GeneralCategory
```

data type

```
---- Char (Read, Show, Eq, Ord, Enum, Bounded)
--
-- unicode
'a'
'\97'        -- codepoint
-- ASCII
'\NUL'       -- 0
'\SOL'       -- 1
'\NUL'       -- 2
'\SOH'       -- 3
-- Unicode
'あ'
'\12354'     -- same

-- Enum Char
['\NUL', '\255']     -- "\NUL\SOH ... abc ... \255"
-- Enum to convert
fromEnum('a')        -- 97. code point
ord 'a'              -- same
toEnum(97) :: Char   -- 'a'
chr 97               -- same


---- String type
-- Just a list of Char
"ABC"
['A', 'B', 'C']        -- same
"Hello\                -- multiline i.e. "HelloThis isA
test"
\This is\
\A test"
```

Utilities

```
---- String Utils
--
import Data.List
lines :: String -> [String]       -- split string by new
line i.e. "\n"
lines "a\nb"       -- ["a","b"]
```

```
words :: String -> [String]       -- split string by
whitespace
words "a b"         -- ["a","b"]
unlines :: [String] -> String     -- inverse. i.e. join
by newline
unwords :: [String] -> String     -- join by whitespace

-- query
isSubsequenceOf :: Eq a => [a] -> [a] -> Bool
"cde" `isSubsequenceOf` "abcdef"


-- General split/join
intercalate :: [a] -> [[a]] -> [a]    -- join
intercalate "->" ["ab","cd",ef"]      -- String join
"ab->cd->ef"
```

```
-- Haskell has no builtin general split
split :: Eq a => a -> [a] -> [[a]]
split c str = if str == []
    then []
    else let
     w = takeWhile (/=c) str
     ws = drop ((length w)+1) str
     in w:(split c ws)
```

```
---- Data.Char utilities
--
-- Ref: https://hackage.haskell.org/package/base/docs/
Data-Char.html
import Data.Char

-- code point
ord :: Char -> Int
chr :: Int -> Char
ord 'a'          -- 49
chr 49           -- 'a'

-- casting
digitToInt :: Char -> Int
intToDigit :: Int -> Char
digitToInt '1'     -- 1
digitToInt 'a'     -- 0xa
digitToInt 'g'     -- ERROR. Not hex
intToDigit 1       -- '1'
intToDigit 0xf     -- 'f'
intToDigit 16      -- ERROR
```

==========
======

Show

GHC.Show: https://hackage.haskell.org/package/base/docs/GHC-Show.html#v:showsPrec

Show

```
-- GHC.Show
class Show a where
  showsPrec :: Int -> a -> ShowS
  show :: a -> String
  showList :: [a] -> ShowS
type ShowS = String -> String

shows :: Show a => a -> ShowS
showChar :: Char -> ShowS
showString :: String -> ShowS
showMultiLineString :: String -> [String]
showParen :: Bool -> ShowS -> ShowS
showList__ :: (a -> ShowS) -> [a] -> ShowS
showCommaSpace :: ShowS
showSpace :: ShowS
showLitChar :: Char -> ShowS
showLitString :: String -> ShowS
protectEsc :: (Char -> Bool) -> ShowS -> ShowS

intToDigit :: Int -> Char
showSignedInt :: Int -> Int -> ShowS
appPrec :: Int
appPrec1 :: Int
asciiTab :: [String]
```

Show
Show is a class for converting a **type** to **string**.
i.e. equivalent of toString() in many languages
Show defines a **string builder ShowS** for the type.

```
type ShowS = String -> String
```

In particular, the string builder is used to convert a nested **data structure** into a string.
i.e. by **recursively** visiting each data in the structure, and **composing** the builder for each item.

Precedence level
A **parent builder** may generate a **child builder** by providing the **parent's precedence level.**
The precedence level is similar to **operator precedence level**.
e.g. with operators, if **parent > child precedence level** then need to add brackets

```
2 + 1 + 1       -- no brackets
2 * (1 + 1)     -- need brackets
```

The string builder precedence level is analogous.
i.e. the child builder may need to change the string it builds by comparing its own precedence with the parent's precedence.
e.g. **wrap** the child in brackets if **parent > child precedence** to **differentiate** the child from the parent.
0 = **lowest precedence** = precedence of **root** parent

e.g. an equation data structure

```
data Equation a = Sum (Equation a) (Equation a) |
Product (Equation a) (Equation a) | Term a

instance Show a => Show (Equation a) where
  showsPrec d (Term x) = shows x
  showsPrec d eq
    | (Sum eq1 eq2) <- eq = showOperator 0 "+" eq1
eq2        -- sum has lowest precedence
    | (Product eq1 eq2) <- eq = showOperator 1 "*" eq1
eq2    -- product has high precedence
    where
      showOperator p symbol eq1 eq2 =
        -- add parenthesis if operator has higher
precedence
        showParen (d > p) $ showsPrec p eq1 . showString
symbol . showsPrec p eq2

-- e.g.
n1 = Term 1
n2 = Term 2
eq1 = Sum n1 (Sum n1 n2)
eq2 = Product n1 (Sum n1 n2)
eq3 = Product n1 (Product n1 n2)
shows eq1 ""      -- "1+1+2"
shows eq2 ""      -- "1*(1+2)"
shows eq3 ""      -- "1*1*2"
```

```
---- Basic
--
builder = shows 123
builder ""            -- "123"
builder "abc"         -- "123abc"
-- compose builders
builder = shows 123 . shows (Just 1) . shows [1]
builder ""                                        --
"123Just 1[1]"

-- showsPrec: create builder
showsPrec :: Show a => Int -> a -> ShowS
shows :: Show a => a -> ShowS
showsPrec 0 (Just 1)       -- precedence 0
shows (Just 1)             -- same
showsPrec 11 (Just 1)      -- precedence 11

-- e.g.
showsPrec 0 (Just 1) ""        -- "Just 1"   i.e. no
brackets
showsPrec 11 (Just 1) ""       -- "(Just 1)" i.e. with
brackets
```

```
showList :: Show a => [a] -> ShowS
showList [1,2,3] ""            -- "[1,2,3]"
---- Utils
--
show :: Show a => a -> String
show 1                -- "1"
shows 1 ""            -- same
-- simple builders
showChar:: Char -> Shows
showString:: String -> ShowS
showParen:: Bool -> ShowS -> ShowS
showChar 'a' "test"            -- "atest"
showString "hello" "test"      -- "hellotest"
showParen True (show 1) ""     -- "(1)"
showParen False (show 1) ""    -- "1"
-- e.g.
showString "value=" . showParen True (shows 1) $
""    -- "value=(1)"
```

==========

Read

GHC.Read: https://hackage.haskell.org/package/base/docs/GHC-Read.html

Read

```
-- GHC.Read
class Read a where
  readsPrec :: Int -> ReadS a
  readList :: ReadS [a]
  readPrec :: ReadPrec a
  readListPrec :: ReadPrec [a]
type ReadS a = String -> [(a, String)]

lex :: ReadS String
lexLitChar :: ReadS String
readLitChar :: ReadS Char
lexDigits :: ReadS String
lexP :: ReadPrec Lexeme
expectP :: Lexeme -> ReadPrec ()
paren :: ReadPrec a -> ReadPrec a
parens :: ReadPrec a -> ReadPrec a
list :: ReadPrec a -> ReadPrec [a]
choose :: [(String, ReadPrec a)] -> ReadPrec a
readListDefault :: Read a => ReadS [a]
readListPrecDefault :: Read a => ReadPrec [a]
readNumber :: Num a => (Lexeme -> ReadPrec a) ->
ReadPrec a
readField :: String -> ReadPrec a -> ReadPrec a
readFieldHash :: String -> ReadPrec a -> ReadPrec a
readSymField :: String -> ReadPrec a -> ReadPrec a
readParen :: Bool -> ReadS a -> ReadS a
```

Read
Read is a class for parsing a string to a given type
i.e. equivalent of parse*() in many languages.
Read providers a **string parser ReadS**

```
type ReadS a = String -> [(a,String)]
```

e.g. Equation parser

```
data Equation a = Sum (Equation a) (Equation a) |
Product (Equation a) (Equation a) | Term a

instance Read a => Read (Equation a) where
  readsPrec d text
    | ((firstEq, rest) : _) <- parsedFirstEq =
readNextLeftAssociative firstEq d rest
    | otherwise = []
    where
      parsedFirstEq = optimalParse $ readFirstEquation d
text -- ++ asParenEquation readFirstUnparenEquation text

      -- 2 possibilities for first equation:
      -- a) a single term
      --    e.g. 1 + ...
      -- b) a parenthesized complete equation
      --    e.g. (1+2) + ...
      --    e.g. (1+2*(3+4)) + ...
      --    e.g. (1) + ...
      readFirstEquation _d _text = readFirstTerm _d
_text ++ readFirstParenEquation _d _text
      readFirstTerm _d = readFirstEquationWith
(readOneTerm _d) _d
      readFirstParenEquation = readFirstEquationWith
(asParenEquation readsPrec)

      readFirstEquationWith :: ReadS (Equation a) ->
ReadsPrec (Equation a)
      readFirstEquationWith readLeft _d _text = do
        (leftEq, leftRest) <- readLeft _text
        readNextLeftAssociative leftEq _d leftRest

      readNextLeftAssociative :: Equation a -> ReadsPrec
(Equation a)
      readNextLeftAssociative leftEq _d "" = [(leftEq,
"")]
      readNextLeftAssociative leftEq _d _text =
        do
          -- case: read next term
          -- 3 possibilities:
          -- a) next term is an operation and left
associative with the parent equation
          --    e.g. 1+1+
          --    e.g. 1+1*
          -- b) next term is an operation but not left
```

```
associative with the parent equation
          --    e.g. 1*1+
          -- c) next term is not an operation
          (op, oprest) <- lex _text
          if isLeftAssociativeOperation _d op
            then do
              let opPrec = getOperationPrecedence op
              (rightEq, rightrest) <- readsPrec opPrec
oprest
              return (applyOperation op leftEq rightEq,
rightrest)
            else return (leftEq, _text)

      -- note: precedence=0
      -- Equation in parenthesis is independent of
parent equation.
      -- Reset precedence to 0.
      asParenEquation :: ReadsPrec (Equation a) -> ReadS
(Equation a)
      asParenEquation s = readParen True (s 0)

      readOneTerm :: ReadsPrec (Equation a)
      readOneTerm _d _text = first Term <$> readValue _d
_text
      readValue :: ReadsPrec a
      readValue = readsPrec

      isLeftAssociativeOperation _d op = isOperation op
&& getOperationPrecedence op >= _d
      isOperation op = op == "+" || op == "*"
      applyOperation op eq1 eq2 | op == "+" = Sum eq1
eq2 | otherwise = Product eq1 eq2
      getOperationPrecedence op | op == "+" = 0 |
otherwise = 1

-- e.g.
parse :: String -> Equation Int
parse = read
parse "1"            -- Term 1
parse "1+1"          -- Sum (Term 1) (Term 1)
parse "2*1+1"        -- Sum (Product (Term 2) (Term 1))
(Term 1)
parse "2*(1+1)"      -- Product (Term 2) (Sum (Term 1)
(Term 1))
```

```
---- Read
--
parser = readsPrec 0 :: ReadS Int
parser "10"            -- [(10,"")]
parser "10a"           -- [(10,"a")]
parser "a10"           -- []. Empty if fail to parsed.

-- readsPrec: create parser with precedence
```

```
readsPrec :: Read a => Int -> ReadS a
reads :: Read a => ReadS a
(readsPrec 0) :: ReadS Int     -- integer parser with
precedence=0
(readsPrec 11) :: ReadS Int    -- precedence=11
readsPrec 0
reads                        -- same

--
readList :: Read a => ReadS [a]
parser = (readList :: ReadS [Int])
parser "[1,2,3]"             -- [([1,2,3],"")]
parser "[1,2,3]a"            -- [([1,2,3],"a"]


-- Read utils
--
read :: Read a => String -> a
(read "123") :: Int
(reads :: ReadS Int) "123"
(read """) :: Int              -- ERROR if it fails to
parse
(read "123a") :: Int           -- ERROR if it fails to
parse

-- readParen:
readParen :: Read a => Bool -> ReadS a -> ReadS a
readParen True (reads :: ReadSInt) "(1)"        --
[(1,"")] . Parenthesis required if True
readParen True (reads :: ReadSInt) "1"          -- []
readParen False (reads :: ReadSInt) "1"         --
[(1,"")]. Parenthesis optional if False
readParen False (reads :: ReadSInt) "(1)"       --
[(1,""),(1,"")].


-- lex: just reads a single word
lex :: ReadS String
lex "ab"               -- [("ab","")]
lex "   ab"            -- [("ab","")]. Initial spaces
trimmed
lex "ab   "            -- [("ab","   ")]. Trailing
spaces removed
lex "ab cd ef"         -- [("ab", " cd ef")]. Multiple
lex ""                 -- [("","")] Empty if empty
string.
lex "   "              -- [("","")] Empty if only spaces
--
```